```
  ___                         __   ___ _
 / __|__ _ _ __  ___   ___ / _| | _ (_)__ _
| (_ / _` | '  \/ -_) / _ \  _| |  _/ / _` |
 \___\__,_|_|_|_\___| \___/_|   |_| |_\__, |
                                      |___/
```

ONTARIO VIRTUAL SCHOOL
ICS3U UNIT 2 PROJECT

The Game of Pig is a classic children's dice game. There are several versions of it written and posted on the Internet. You are given a skeleton program to complete and must use all of the code given to ensure that you have demonstrated a clear understanding of Unit 2's lessons. In this version of the game you will roll a single die to accumulate a score. The rules of the game are found in the actual python file called Unit2_Project_GameOfPig.py.

Your Programmer's Journal is a vital tool of communicating your knowledge and understanding of the programming process. Each day you should carefully mark down what you accomplished and learned while coding the project. Include the date of each entry, resources you used while exploring the solution, and problems you encountered.

Ministry Expectations

- Demonstrate the ability to use different data types
- Use proper code maintenance techniques
- Use a variety of problem-solving strategies
- Design algorithms according to specifications
- Apply a software development life-cycle model (design-code-test-repeat)
- Demonstrate an understanding of the software development process

Steps

1. Run the code attached to this project: Unit2_Project_GameOfPig.py
2. Understand the given skeleton program and how it works
3. Plan your changes to the existing code – set milestones
4. Generate flow-charts or pseudocode of your changes (add to your journal)
5. Use step-wise-refinement to test your code at each milestone (comment in your journal)
6. Update your programmer's journal at every stage **
7. Complete the *Game Of Pig* project as a human vs. computer player game
8. Ensure that existing code is preserved as much as possible
9. Comment all internal blocks of code as required (replacing teacher comments where needed)
10. Submit your project file and programmer's journal.

Evaluation

See Rubric on page 2

Feedback

Achievement Chart – Computer Science Grade 11 (2020)				
Categories	Level 4 *(80-100%)*	Level 3 *(70-79%)*	Level 2 *(60-69%)*	Level 1 *(50-59%)*
Programmer's Journal (Knowledge / Understanding): The assignments software life cycle is clearly understood and programmer's vocabulary *__ / 20*	The programmer's journal follows all specifications, clearly demonstrates growing knowledge over the course of the project, is authentic in its exploration of the problems to be solved.	The programmer's journal follows all specifications including a timeline, clear milestones (planning), helpful bibliography of resources used, and an authentic exploration of the issues.	The programmer's journal exists and follows most of the specifications, discusses the growth of the project through a timeline following the software life cycle: planning, implementing, etc.	The programmer's journal exists, but the content is either trivial, lacking robustness or fails to demonstrate the programmer's growing body of knowledge in some way.
Solution (Thinking): Flow of Control (FC), and Logic (L) are well formed and function together to create a fully functional program *__ / 20*	FC exists without flaws, such that normal edge case input produces expected output. L is such that the program works for expected and unexpected input.	FC exits without flaws, such that normal input and output is correct. L is such that the program works for expected and unexpected input.	FC exists without flaws, such that normal input and output is correct. L is such that the program works for expected input.	FC exists and may be flawed in such a way as to not impact most program operation. L is such that the program may not work in all test cases.
Coding Practice (Application): Data Types (DT), and the decomposition of the program into blocks (Methods / Functions) *__ / 20*	DT clearly organize the code and have useful identifiers and are reused where possible. The program makes full use of modular design making the program flow understandable.	DT are properly named and used in the program. Program uses methods in a logical way that makes the program easier to understand.	Data Types are assigned to variables in a logical manner. The program contains methods which work.	Data Types are used but memory may not be managed or organized in the code. The program contains methods that work but are deficient in some way.
Documentation (Communication): Internal parts of the program, in all associated files are properly commented / documented *__ / 20*	Program files have all parts of the internal documentation including: Header with purpose, author, date; all major blocks commented.	All major parts of the code are preceded with a comment: loops, if-blocks, sections of input, process, output. These sections are clearly meaningful adding clarity to the code.	Code is self-documenting in terms variable names, method names, and any ambiguous lines of code contain internal commenting to clarify purpose.	Some attempt at internal documentation is present in the program code. There is, however, clearly missing documentation or little attempt at properly naming methods or variables.